# Planning & Reinforcement Learning

UniVR - Dipartimento di Informatica

**Tommaso Toffali**

1° Semestre 2026/2027

# Indice

# Parte I
# Classical Planning

## 1 Introduction to Planning

### 1.1 Historical Context and Motivation

Planning has been a central topic in artificial intelligence since the inception of the field at the **Dartmouth Conference in 1956**, where the foundings fathers of AI first gathered to define the scope and ambitions of this new discipline. Planning represent one of the foundamental capabilities required for intelligence behavior: the ability to reason about sequences of actions that achieve desired goals.

The motivation for studying automated planning are manifold:

- **Autonomous agents**: Enabling robots, software agents, and autonomouse system to make decision and act independently in complex environemnts
- **Resource optimization**: Finding optimal or near-optimal stragies for Resource allocation, scheduling, and logistics
- **Scientific applications**: Modeling and solving problems in domain such as space exploration, manufacturing, and healtcare
- **Theoretical foundations**: Understanding the computional complexity and formal properties of reasoning about action and change

### 1.2 The planning Cycle

The complete planning process involves several interconnected pahses that form a continuos cycle:

1. **Plan Generation**: Given a description of the current state, available actions, and desired goals, synthesize a sequence of actions (a plan) that transforms the inital state into a goal state. This phase involves:
    - Analyzing the initial state to understand what is currently true
    - Identifying the gap between the current state and the goal state
    - Searching throungh the space of possible action sequences
    - Evaluating candidate plans for correctness and optimality
    - Selecting the best plan according to specified criteria (e.g., shortes length, minimum cost, fastest execution)
2. **Plan Deployment**: Execute the generated plan in the actual environment, monitoring the execution to detect deviations from expected behavior. This phase includes:
    - Translating abstract plan actions into concrete executable commands
    - Monitoring sensors and feedback to verify that actions have the expected effects

- Detecting discrepancies between predicted and actual states
- Maintaining a record of executed actions for debugging and learning

3. **Replanning**: When execution monitoring detects that the current plan is no longer valid (due to unexpected events, actions failures, or environmental changes), generate a new plan or repair the existing one. Replanning strategies includes:
   - **Plan repair**: Modify the existing plan minimally to accomodate the new situation
   - **Replan from scratch**: Generate an entirely new plan from the current state
   - **Contingency planning**: Use pre-computed alternative plans for anticipated failures

This cycle reflects the reality that planning systems must operate in dynamic, partially unpredictable environments where initial plans may need adaptation. The cyle continues iteratively until the goal is achived or deemed unreachable.

### 1.2.1 Challenges in Real-World Planning

Real-World deployment of planning systems faces several challengs:

- **Excution uncertainty**: Actions may fail or have unexpected outcomes
- **Incomplete information**: The planner may not have complete knowledge of the environment
- **Dynamic environments**: The world may change while the plan is beign executed
- **Computional constraints**: Planning must often occur in real-time with limited computational resoruces
- **Plan quality vs. planning time**: Trade-off between finding optimal plans and responding quickly

These challenge motivate the development of robust planning algorihms that cna handle unvertainty, adapt to changes, and operate efficiently under resource constraints.

# 2 Formalization of the Planning Problem

## 2.1 State Transition Systems

The mathematical foundation of planning rests on the concept of state transition system, which provides a formal model of how actions transform states.

**Definizione 2.1 (State Transition System)**

state transition system is a tuple $\Sigma = (S, A, \gamma)$ where:

- $S$ is a finite or countably infinite set of **states**

- $A$ is a finite or countably infinite set of **actions**
- $\gamma : S \times A \to S$ is a **state transition function** that maps a state and an action to a resulting state

The transition function $\gamma(s, a) = s'$ specifies that executing action $a$ in state $s$ result in state $s'$. This function encodes the **dynamics** of the domain - how the world changes in response to actions.

### 2.1.1 Properties of State Transition System

State transition systems can be characterized by serveral important properties:

- **Determinism**: In a deterministic system, $\gamma$ is a function - each state-action pair leads to exactly one successor state. In non-deterministic systems, multiple outcomes are possible.
- **Reachability**: A state $s'$ is reachable from state $s$ if there exists a sequence of actions that transform $s$ into $s'$. The reachable state space from an initial state is often much smaller thant the totale state sapce.
- **Reversibility**: An action is reversible if there exists another action that undoes its effects. Many real-world actions are irreversible (e.g., breaking an object)
- **State space structure**: The connectivity and topology of the sate space significanyly affect planning complexity. Highly connected spaces may have many solution paths, while sparse spaces may have few or no solutions.

## 2.2 Classical Planning: Foundamental Assumptions

Classical planning makes several simplyfying assumptions that restrict the class of the problems considered but enable efficient algorithmic solutions.

**Definizione 2.2 (Classical Planning Framework)**

he classical planning framework relies on six key assumptions:

1. Finitely many states
2. Finitely many actions
3. Deterministic transition function
4. Full observability
5. Instantaneous actions
6. No exogenous events

1. **Finitely many states**: The state space $S$ is finite, allowing exhaustive search techniques. This assumption ensures that the planning is decidable and that search alogirhms will terminate

   *Jusitfication*: While real-world domains may have infinite state spaces (e.g.

continuos variables), finite approximations are often sufficient for pratical purposes.

2. **Finitely many actions**: The action space $A$ is finite, ensuring decidability. Each state has a finite branching factor in the state space graph.

   *Jusification*: Even in complex domains, the number of distinct action type is typically manageable, though the number of ground actions (instantieted with specific objects) may be LARGE

3. **Deterministic transition function**: For every state $s$ and action $a$, there is exactlu one resulting state $\gamma(s, a)$. Actions have predictable, certain outcomes.

   *Justification*: Determinism simplified reasoning about action effects. Nondeterministic planning requires more complex formalism (e.g., Markov Decision Processes)

4. **Full observability**: The planner has complete knowledge of the current state at all times. There is no uncertainty about which state the system occupies.

   *Justification*: Full observability eliminates the need for belief state reasoning and sensing actions. Partial observability requires more sophisticated planning approaches.

5. **Instantaneous actions**: Actions have no duration, they occur instantaneously. Temporal reasoning is not required.

   *Justification*: Ignoring action durations simplifies the planning model. Temporal planning extends classical planning to handle durations and concurrent actions.

6. **No exogenous events**: Only the planning agent can change the world through deliberate actions. The environment remains static unless acted upon.

   *Justification*: Exogenous events (e.g., other agents, natural processes) introduce additional complexity. Classical planning assumes a static world between actions.

These assumptions, while restrective, capture a significant and important class of planning problems and provide a foundation for understanding more complex, realistic planning scenarios.

### 2.2.1 Relaxing Classical Assumptions

Modern planning research has devoloped extensions that relax these assumptions:

- **Probabilistic planning**: Handles non-deterministic actions with probability distributions over outcomes
- **Conformant planning**: Plans under partial observability without sensing

- **Contingent planning**: Generates conditional plans that include sensing actions
- **Temporal planning**: Reasons about actions durations and temporal constraints
- **Multi-agent planning**: Coordinates plans among multiple agents

## 2.3 The Planning Problem

Given a state transition system $\Sigma = (S, A, \gamma)$ we can now formally define the planning problem.

**Definizione 2.3 (Planning Problem)**

planning problem is a tuple $P = (\Sigma, s_0, G)$ where:

- $\Sigma = (S, A, \gamma)$ is a state transition system
- $s_0 \in S$ in the **initial state**
- $G \subseteq S$ is a set of **goal states**

Alternatively, goals can be specified as a **goal formula** $g$, a logical expression that is satisfied by exactly those states in $G$. This allows more compact and expressive goal specifications.

### 2.3.1 Goal specification

Goals can be specified in several ways:

- **Explicit goal states**: Enumerate the set $G$ of accetable final states. This is impratical for large state spaces.
- **Goal formula**: A logical formula $g$ such that $G = s \in S \mid s \models g$. For example, in propositional logic : $g = At(Robot, RoomB) \wedge Clean(RoomB)$
- **Goal conditions**: A set of propositions that must be true in the goal state. This is the most common approach in classical planning.
- **Utility functions**: In optimization settings, goals may be specified as utility functions to maximize or cost functions to minimize.

**Definizione 2.4 (Solution Plan)**

**solution** to a planning problem $P = (\Sigma, s_0, G)$ is a sequence of actions $\pi = (a_1, a_2, \ldots, a_n)$ such that:

$$s_n = \gamma(\gamma(\ldots\gamma(\gamma(s_0, a_1), a_2)\ldots, a_{n-1}), a_n) \in G$$

That is, executing the sequence of actions starting from the initial state $s_0$ results in a goal state.

We can also write this more compactly using the notation $\gamma^*(s, \pi)$ to denote the state reached by executing plan $\pi$ from state s:

$$\gamma^*(s_0, \pi) \in G$$

### 2.3.2 Plan Quality

Not all solution plans are equally desirable. Common quality metrics include:

- **Plan length**: Number of actions in the plan. Shorter plans are often preferred.
- **Plan cost**: Sum of actions costs. Each action $a$ may have an associated cost $c(a)$.
- **Makespan**: Total execution time (relevant in temporal planning).
- **Resource consumption**: Amount of resources used during execution.

An **optimal plan** minimizes the chosen quality metric among all solutions plans.

# 3 Representation of Planning Problems

## 3.1 The challenge of explicit representation

While the state transition system formalism is mathematically elegant, it faces a critical pratical challenge: **the curse of dimensionality**. Real-world planning domains can have exponentially large sate spaces. For example:

- A domain with $n$ boolean variables has $2^n$ possible states
- A domain with $k$ objects and $m$ binary relations has up to $2^{mk^2}$ states
- Enumerating all states and transitions explicitly is infeansible for even moderately-sized problems

**Esempio 3.1 (Blocks World Complexity)**

onsider a Blocks World domain with $n$ blocks. The number of possible configurations grows super-exponentially with $n$. For $n = 10$ blocks, there are more than $10^13$ possible configurations. Explicitly representing the transition function would require storing information about trillions of state-action pairs.

This motivates the need for **compact, structured representation** that exploit regularities in the domain to represent large state spaces and transition functions implicitly.

## 3.2 Action Schemas: Structured Representation

The classical approach to compact representation uses **action schemas** (also called action template or operators). An action schema is a prametrized description of a family of related actions.

**Definizione 3.2 (Action Schema)**

n action schema consists of:

- **Name and parameters**: A symbolic name and typed parameters, e.g., $Move(x, y)$.
- **Preconditions**: A logical formula Pre specifying when the action can be executed
- **Effects**: A logical formula $Eff$ specifying how the sate changes when the action in executed

**Actions** are **ground instances** of action schemas, obtained by binding the parameters to specific objects in the domain. For example, the schema $Move(x, y)$ might generate ground actions $Move(RoomA, RoomB)$, $Move(RoomB, RoomC)$, etc.

### 3.2.1 Advantages of Action Schemas

Action schemas provide severeal benefits:

- **Compactness**: A single schema can represent exponentially many ground actions
- **Generality**: Schemas capture the general structure of actions independent of specific objects
- **Scalability**: Adding new objects to the domain automatically generates new appliable actions
- **Knowledge reuse**: Schemas learned in one domain can be transferred to similar domains.

## 3.3 State Representation: Propositional logic

In classical planning, states are typically represented using **propositional logic**:

- A state is a set of **atoms** (propositional variables) that are true in that state
- Atoms not in the set are assumed false (**closed-world assumption**)
- Action preconditions and effects are logical formula over these atoms

The representation allows:

- Compact encoding of structured states
- Efficient reasoning about action applicability and effects
- Use of logical inference techniques for plan generation

### 3.3.1 State Update Semantics

**Esempio 3.3 (STRIPS Assumption)**

hen an action is executed, the state is updated according to the action's effects:

- **Add list**: Atoms that become true after the action
- **Delete list**: Atoms that become false after the action
- **Frame axioms**: Atoms not mentioned in the effects remain unchanged

The **STRIPS assumption** (named after the Stanford Research Institute Problem Solver) states that only atoms explicitly mentioned in the effects changem all other atoms persist unchanged. This simplifies reasoning about action effects.

## 3.4 Example: Blocks World

Consider the classic **Blocks World** domain, one of the most studied domain in planning research

### 3.4.1 Domain Description

The Blocks World consists of:

- A set of blocks that can be stacked on top of each other
- A table with unlimited space
- A robot arm that can pick up and put down blocks

**Esempio 3.4 (Blocks World State Representation)**

State predicates:

- $\texttt{On}(x, y)$: Block $x$ is directly on top of blocks $y$.
- $\texttt{OnTable}(x)$: Block $x$ is on the table
- $\texttt{Clear}(x)$: Block $x$ has no blocks on top of it
- $\texttt{Holding}(x)$: The robot arm is holding block $x$.
- $\texttt{ArmEmpty}$: The robot arm is not holding anything

### 3.4.2 Action Schemas

**PickUp**(x): Pick up block $x$ from the table

Parameters: $x$(block)
Precondition: $\texttt{Clear}(x) \land \texttt{OnTable}(x) \land \texttt{ArmEmpty}$
Effect: $\texttt{Holding}(x) \land \neg\texttt{OnTable}(x) \land \neg\texttt{Clear}(x) \land \neg\texttt{ArmEmpty}$

**PutDown**(x): Put down block $x$ on the table

Parameters: $x$(block)
Preconditions: $\texttt{Holding}(x)$
Effect: $\texttt{OnTable}(x) \land \texttt{Clear}(x) \land \texttt{ArmEmpty} \land \neg\texttt{Holding}(x)$

**Stack**(x, y): Stack block $x$ on top of block $y$.

Parameters: $x, y$(blocks)
Preconditions: $\texttt{Holding}(x) \land \texttt{Clear}(x)$
Effect: $\texttt{On}(x, y) \land \texttt{Clear}(x) \land \texttt{ArmEmpty} \land \neg\texttt{Holding}(x) \land \neg\texttt{Clear}(y)$

**Unstack**(x, y): Remove block $x$ from on top of block $y$.

Parameters: $x, y$(block)
Precondition: $\texttt{Holding}(x) \land \texttt{Clear}(x) \land \texttt{ArmEmpty}$
Effect: $\texttt{Holding}(x) \land \texttt{Clear}(y) \land \neg\texttt{On}(x, y) \land \neg\texttt{Clear}(x) \land \neg\texttt{ArmEmpty}$

#### 3.4.3 Example Problem Instance

**Initial state**: Blocks A, B, C are all on the table

$s_0 = \{\texttt{OnTable}(A), \texttt{OnTable}(B), \texttt{OnTable}(C), \texttt{Clear}(A), \texttt{Clear}(B), \texttt{Clear}(C), \texttt{ArmEmpty}\}$

**Goal**: Stack the blocks in the order A on B on C

$$G = \{\texttt{On}(A, B), \texttt{On}(B, C), \texttt{OnTable}(C)\}$$

**Solution plan**:

1. `PickUp`(B)
2. `Stack`(B,C)
3. `PickUp`(A)
4. `Stack`(A,B)

# Parte II
# Advanced Planning Representation

## 4 Reduction to Propositional Satisfiability

### 4.1 Motivation

We continue our discussion of the formalization and representation of planning problems. A fundamental technique in automated planning is to **reduce the planning problem to the stisfiability problem of propositional logic**. This reduction allows us to leverage powerful SAT solvers to find plans efficently.

**Definizione 4.1 (Reduction to SAT)**

he key idea is to encode:

- The initial state as a propositional formula
- The goal conditions as a propositional formula
- The action effects and preconditions as propositional formulas
- The constraint that actions must form a valid sequence

If the resulting formula is satisfiable, the satisfying assignment corresponds to a valid plan.

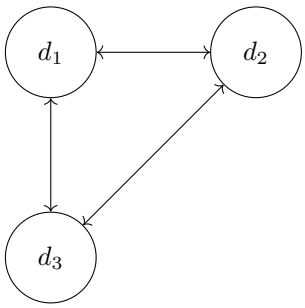
## 4.2 Example Domain: Dock Worker Robots (DWR)

To illustrate these concepts, we introduce the **Dock Worker Robots (DWR)** domain, a classic planning benchmark that models container logistics at a port.

### 4.2.1 Domain description

The DWR world consists of:

- **Docks**: Three docks $d_1, d_2, d_3$ arranged in a triangular configuration
- **Connectivity**: Docks are connected as follows: $d_1 \leftrightarrow d_3, d_1 \leftrightarrow d_2, d_2 \leftrightarrow d_3$.
- **Piles**: Storage locations at docks where containers can be stacked
- **Containers**: Object that need to be moved between locations
- **Robots**: Mobile agents that can move containers between piles




## 4.3 Initial State configuration

The initial state of our example problem is:

- **At dock** $d_1$: Pile $p_1$ contains two containers: $c_1$ (on top) and $c_2$ (below)
- **At dock** $d_2$:
  - Pile $p_2$ contains one container: $c_3$.
  - Pile $p_3$ is empty

- **At dock** $d_3$: Pile $p_4$ is empty

RIPRENDERE APPUNTI FINO A SLIDE 46

# 5 Conflict-Driven Clause Learning (CDCL)

Modern SAT solvers use **Conflict -Driven Clause Learning (CDCL)**, an advanced technique that combines depth-first search with intelligence learning from conflicts to efficently solve large satisfiability problems.

## 5.1 CDCL Algorithm Overview

CDCL implements **depth-first search strategy** with intelligent backjumping (undo one or more decision) and clause learning:

**Definizione 5.1 (CDCL Components)**

**CDCL** combines several key techniques:

- **Unit propagation**: Efficient inference of forced assignments
- **Conflict analysis**: Learning from failures to guide future search
- **Backjumping**: Intelligent backtracking to erliaer decision levels
- **Clause learning**: Adding learned constraints to prevent repeated conflicts

## 5.2 Backjumping: Generalized Backtracking

Backjumping is a generalization of the basic backtracking (undo one decison) mechanism that allows the solver to jump back to erlier decision levels, not just the immediately preceding one.

**Definizione 5.2 (Backjumping vs Backtracking)**

In **backjumping**, when a conflict occurs, the solver analyzes the conflict cause to determine the earliest decisione level that contributed to conflict. It then jumps back to that level, rather than just the immediately preceding decision.

**Advantage**: Backjumping can skip over irrelevant decision levels, significantly reducing the search space compared to basic backtracking.

### 5.2.1 Conflict analysis example

COnsider a conflict clause $C = L_1 \vee \ldots \vee L_k$ where all literals are currently false. The analysis determines:

- Why each literal $L_i$ is false in the current trail $\Gamma$.
- The decision level at which each literal became false

- The erliest decision level that contributed to the conflict

The solver then backjumps to that earliest relevant decision level, effectively skipping decision levels that did not contribute to the conflict.

# 6 Binary Resolution

The theoretical foundation of conflict analysis relies on **binary resolution**, a foundamental inference rule in propositional logic

## 6.1 Resolution Rule

**Definizione 6.1 (Binary Resolution)**

Given two clauses containing complementary literals:

$$\frac{C \vee L \quad D \vee \neg L}{(C \vee D)}$$

The **resolvent** $C \vee D$ is logically entailed by the two premises. This rule allows deriving new clauses that preserve satisfiability.

## 6.2 Soundness of Resolution

Binary resolution is **sound**, the resolvent is logically entailed by the permise.

**Teorema 6.2 (Resolution Soundness)**

$If C \vee L, D \vee \neg L \models R where R is any formula, then C \vee L, D \vee \neg L \models (C \vee D)$.

*Proof.* Consider any interpretation $v$:

- $If v \models L$, then since $v \models D \vee \neg L$, we must have $v \models D$. Since $v \models C \vee L$, we have $v \models C \vee D$.
- $If v \models \neg L$ then since $v \models C \vee L$, we must have $v \models C$. Since $v \models D \vee \neg L$, we have $v \models C \vee D$.

In both cases, $v \models C \vee D$, so the resolvent is logically entailed by the premises.

## 6.3 Application in Conflict Analysis

In conflict analysis, resolution is used to explain why conflicts occur and to determine which decision levels are responsible:

- When a literal $L_k$ in the conflict clause is false due to propagation (not a decision)
- The propagation was justified by a clause $D \vee \neg L_k$.
- By resolving the conflict clause with the justification clause, we can trace back the decision that ultimately caused the conflict.

# 7 Explanation Mechanism

The explanation mechenism in CDCL traces the implications that led to a conflict, allowing the solver to learn more general constraints.

**Teorema 7.1 (Explanation rule)**

$$\langle S \| \Gamma \| C \rangle \rightarrow \langle S \| \Gamma \| C \vee D \rangle$$

if $\neg L \in \Gamma$ where $\neg L$ was propagated using the clause $D \vee \neg L$.

## 7.1 Explanation Process

The explanation mechanism works as follows:

1. Start with a conflict clause $C = L_1 \vee \ldots \vee L_k$.
2. For each literal $L_i$ that is false due to propagation (not a decision):
   - Find the justification clause that forced $\neg L_i$ into the trail
   - Resolve the current conflict clause with the justification clause
   - Continue untile all literals in the conflict clause are explained by decision
3. The final resolved clause contains only literals that were decided (not propagated)
4. This clause can be learned and added to the formula to prevent similar conflicts

# Parte III
# SAT-Based Techniques

# 8 Modern SAT Solving

## 8.1 CDCL: An evolution of DPLL

CDCL (**Conflict-Driven Clause Learning**) represent an evolution of the DPLL decision procedure for SAT. While CDCL maintains the same basic decision rule as DPLL, it introduces sophisticated conflict anaysis and learning mechanisms that dramatically improve solver efficiency on real-world instances.

### 8.1.1 CDCL Transition Rules

The CDCL procedure employs the following transition rules in search mode: This four rules define the behavior of CDCL in **search mode**, which is identical to the DPLL procedure. The key difference lies in how CDCL handles conflicts through its conflict analysis and learning machanisms.

## 8.2 Decision Levels in CDCL

The trail $\Gamma$ in CDCL is organized into **decision levels** that provides structure ti the search process:

- The trail begins at **decision level 0** (empty trail)
- Each decision literal opens a new decision level
- If the maximum decision level in $\Gamma$ is $n$, the next decision opens level $n + 1$.
- All propagations following a decision remain at the same level untile the next decision

### 8.2.1 Trail as a Stack Structure

The trail can be conceptualized as a **stack structure**:

1. Initially, the stack is empty (decision level 0)
2. A decision literal is puhsed onto the stack, opening a new level
3. Unit propagations following the decision are pushed onto the stack at the same level
4. Another decision is maade, opening the next level
5. The process continues with alternating decision and propagations
6. During backtracking/backjumping, literals are popped from the stack until reaching the target decision level

This organization enables efficient backjumping by allowing the solver to quickly identify and return to relevant decision levels during conflict analysis.

## 8.3 Illustrative example

**Esempio 8.5 (Instance)**

$S = \{P, \neg P \vee Q \vee R, T \vee \neg Q, \neg R \vee Q\}$

The inital propagation and decision sequence proceeds as follows·

$$\langle S \| \epsilon \rangle \xrightarrow{prop} \langle S \| P_P \rangle$$